

LangChain应用框架实践教程

从基础到实践，构建你的第一个LLM应用

我们将遵循以下蓝图构建应用



基础构件 (The Foundation)

深入理解LangChain的核心价值与整体生态，并完成开发环境的配置。



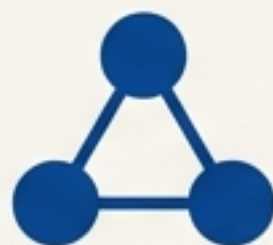
核心引擎 (The Core Engine)

掌握模型I/O，学习与大语言模型进行标准且高效交互的最佳实践。



知识中枢 (The Knowledge Hub)

学习数据检索，让模型能够连接并利用你的外部私有数据。



整合展望 (The Big Picture)

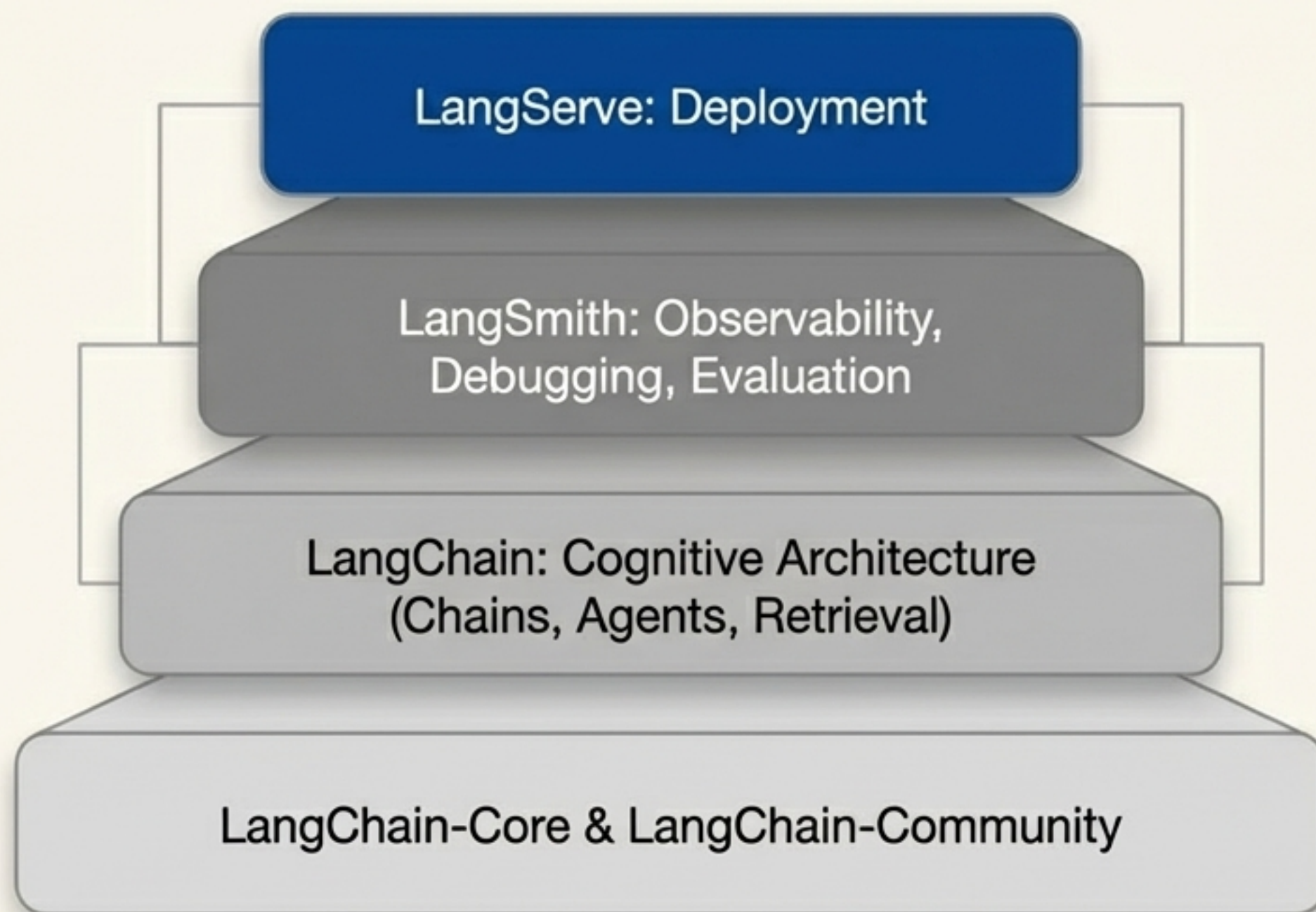
回顾我们构建的应用架构，并探索LangChain更高级的功能。

为何需要LangChain? 因为它为复杂性提供了标准化的解决方案

构建一个简单的LLM调用很容易, 但构建一个可维护、可扩展、具备状态的应用却很复杂。
LangChain提供了一套标准化的、可组合的构件来管理这种复杂性。



LangChain不只是一个库，而是一个完整的生态系统



从开发、调试、评估到部署，LangChain生态系统为LLM应用的整个生命周期提供支持。

动手第一步：安装核心库并配置密钥

安装 (Installation)

首先，我们需要安装LangChain的核心库以及OpenAI的集成库。

```
# Install core LangChain packages  
pip install langchain  
pip install langchain-openai
```

配置 (Configuration)

然后，将你的OpenAI API密钥设置为环境变量，以便LangChain能够访问模型。

```
# On macOS/Linux  
export OPENAI_API_KEY="your-api-key-here"  
  
# On Windows (Command Prompt)  
set OPENAI_API_KEY="your-api-key-here"
```


模型I/O：与LLM对话的标准流程



LangChain的Model I/O将与模型的交互抽象为三个步骤：使用**Prompts**格式化输入，将请求发送给**Models**，并用**Output Parsers**解析输出。

实战：编写你的第一个动态LLM调用

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

# 1. 初始化模型
model = ChatOpenAI(model="gpt-4")

# 2. 创建一个可复用的提示模板
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful translator."),
    ("human", "Translate this text from {input_language} to {output_language}: {text}")
])

# 3. 将它们链接在一起形成一个可执行链
chain = prompt | model

# 4. 调用链并传入参数
chain.invoke({
    "input_language": "English",
    "output_language": "Chinese",
    "text": "I love programming."
})
```

通过将提示和模型链接（`|`），我们创建了一个可复用的、类型安全的执行链。

超越简单问答：流式输出与函数调用

流式输出 (Streaming)

使用 `.stream()` 方法可以像打字机一样逐个字符地获取输出，极大提升用户体验。

```
for chunk in chain.stream({"text": "写一首关于AI的短诗"}):  
    print(chunk.content, end="", flush=True)
```

函数调用 (Function Calling)

让LLM能够调用外部工具或函数，赋予它执行实际操作的能力。

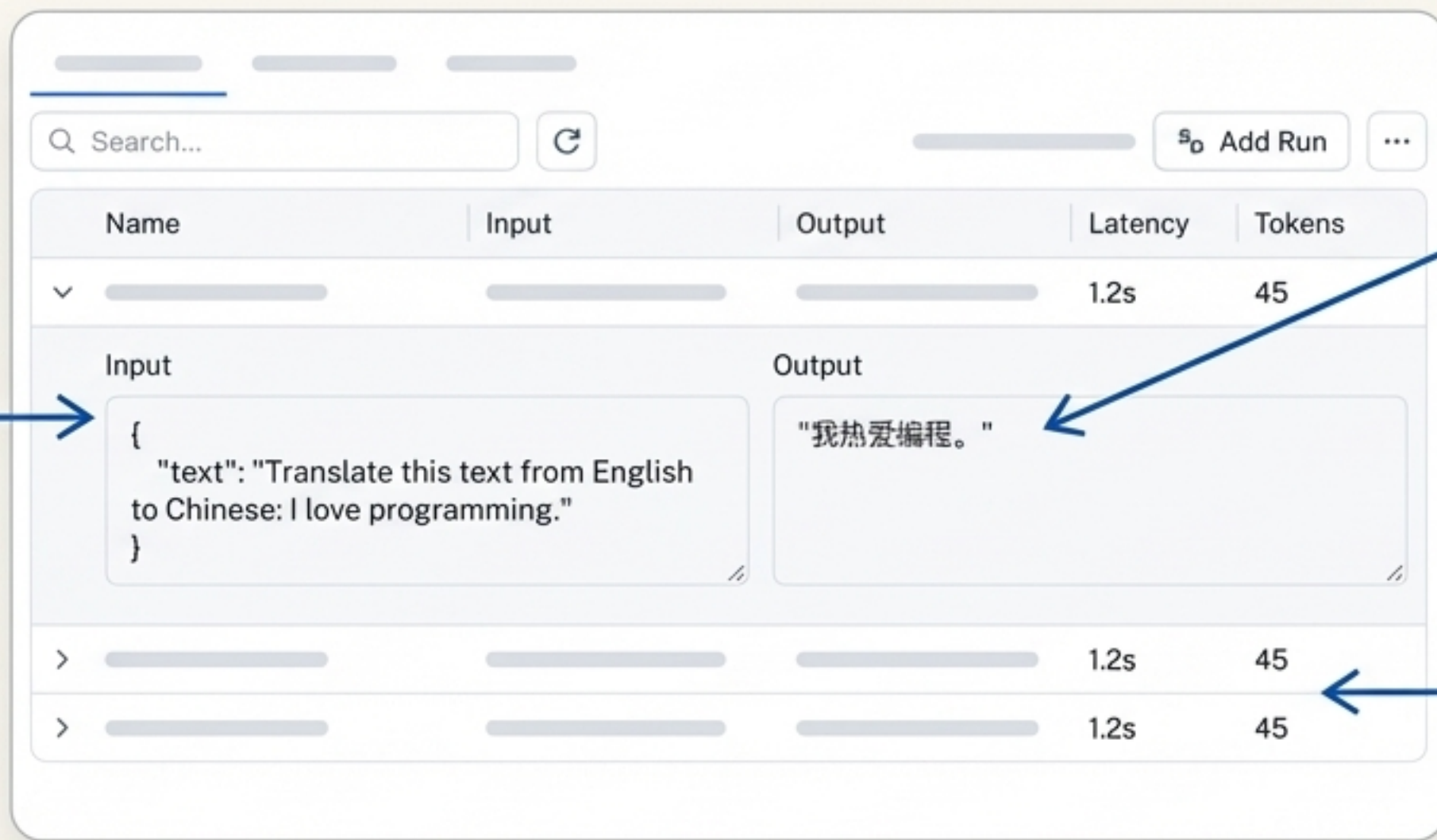
```
from langchain_core.utils.function_calling import  
convert_to_openai_tool  
  
def multiply(a: int, b: int) -> int:  
    """Multiplies two integers together."""  
    return a * b  
  
multiply_tool = convert_to_openai_tool(multiply)  
model_with_tool = model.bind_tools([multiply_tool])
```


揭开黑盒：使用LangSmith实现应用的可观测性

只需设置几个环境变量，即可自动追踪你应用中的每一次LLM调用。

```
export LANGCHAIN_TRACING_V2="true"
export LANGCHAIN_API_KEY="..."
```

清晰看到传入模型的完整提示

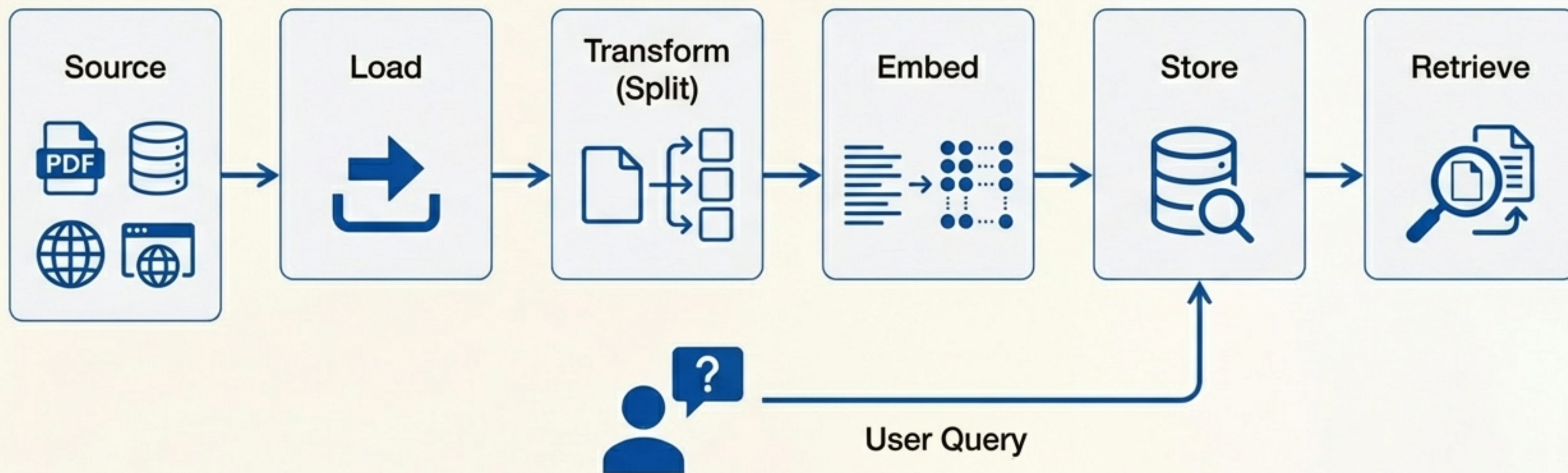


Name	Input	Output	Latency	Tokens
▼			1.2s	45
	<pre>{ "text": "Translate this text from English to Chinese: I love programming." }</pre>	<pre>"我热爱编程。"</pre>		
>			1.2s	45
>			1.2s	45

查看模型的原始输出

监控性能和成本

让LLM学习你的私有数据：检索增强生成（RAG）



RAG的核心思想：在生成答案之前，先从你的知识库中检索最相关的信息，并将其作为上下文提供给LLM。

RAG实战（1/2）：构建知识库的注入管道

Step 1: 加载文档 (Load Document)

```
from langchain_community.document_loaders import TextLoader
loader = TextLoader("my_document.txt")
docs = loader.load()
```

Step 2: 分割文本 (Split Text)

```
from langchain.text_splitter import CharacterTextSplitter
text_splitter = CharacterTextSplitter(chunk_size=500, chunk_overlap=0)
splits = text_splitter.split_documents(docs)
```

Step 3: 嵌入并存储 (Embed and Store)

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
vectorstore = Chroma.from_documents(documents=splits, embedding=OpenAIEmbeddings())
```


RAG实战（2/2）：从知识库中检索并生成答案

Step 4: 创建检索器 (Create Retriever)

将我们的向量数据库转换为一个标准的检索器接口。

```
retriever = vectorstore.as_retriever()
```

Step 5: 调用检索器 (Invoke Retriever)

现在，我们可以用一个问题来调用检索器，它会自动返回最相关的文档片段。

```
query = "小说里三体舰队的结局是什么？"  
retrieved_docs = retriever.invoke(query)  
print(retrieved_docs[0].page_content)
```

Output:

```
[Document(page_content='...两个世纪后，幸存的“蓝色空间”号和“万有引力”号在追击“三体第一舰队”的残余舰队时，意外发现了四维碎块。在进入四维空间后，他们摧毁了三体舰队，但最终决定不返回地球...', metadata={'source': 'my_document.txt'})]
```


架构回顾：我们构建了一个完整的LLM应用原型

应用架构蓝图

基础构件 ✓



我们搭建了一个可运行的开发环境，为所有后续工作奠定了基础。

核心引擎 (Model I/O) ✓



我们实现了一个标准化的模块，用于灵活地构造提示、调用模型，并处理流式输出和工具使用。

知识中枢 (Retrieval) ✓



我们构建了一条完整的数据管道，将外部文档转化为可查询的知识库，成功地将私有数据与LLM连接起来。

这三个部分共同构成了一个功能完备、可观测、可扩展的RAG应用架构。

下一步探索：从组件到自主应用



Chains (链)

将我们今天学习的组件（如检索器、提示、模型）串联起来，构建更复杂的、多步骤的逻辑工作流。



Agents (智能体)

赋予LLM使用工具（如函数调用、检索引擎）进行自主决策和行动的能力，以完成更开放式的任务。

掌握了今天的构建块，你已经准备好开始探索Chains和Agents，构建真正智能的应用。

LangChain 是从 API 调用到生产级 AI 应用的桥梁



你今天学习的不仅是一个库，更是一种构建思维。利用这些模块化、可组合的工具，你可以将任何复杂的AI应用构想，系统化地变为现实。